

```
<?xml version="1.0"?>

<!-- Define the name of ruleset -->
<ruleset name="Project_A Standard">

<!-- Include all standards from PEAR standards -->
<rule ref="PEAR"/>

<!-- Include all standards from Squiz and exclude specific
standard -->
<rule ref="Squiz">
    <exclude name="Squiz.WhiteSpace.PropertyLabelSpacing"/>
</rule>

<!-- Lines can be 80 chars long, show errors at 120 chars
-->
<rule ref="Generic.Files.LineLength">
    <properties>
        <property name="lineLimit" value="80"/>
        <property name="absoluteLineLimit" value="120"/>
    </properties>
</rule>
</ruleset>
```

You can use the above rule-set by using the following command:

```
$phpcs --standard=<ruleset_xml_filepath> <php_file>
```

Creating your own sniffs

A good programmer and a good project can always do with improvement. If you have identified some new rules to follow and if they do not feature among the existing coding sniff collections, you can also write your own.

Here I go over one of my project-specific requirements and the method to create a sniff for it. During development, we tend to use many print functions and local settings that should not be present in production code. Sometimes, we forget to remove these lines on deployment and see some surprises on release. To avoid this situation, in my team, we decided to follow a standard to identify the temporary and local codes. For this, we added the following comment before the temporary code:

```
//test_code
```

For example, in the following file, called sample2.php, we use proxy settings for the development environment, which should not remain in the production environment:

```
<?php
$ch = curl_init();
curl_setopt($ch, CURLOPT_URL, 'www.php.net');
//test_code
```

```
curl_setopt($ch, CURLOPT_PROXY, 'localhost:3128');
curl_setopt($ch, CURLOPT_RETURNTRANSFER, TRUE);
$data = curl_exec($ch);
curl_close($ch);
echo $data;
?>
```

Now, let's write the sniff for the above situation. Before starting to write the sniff, you should understand the file structure of the code sniffer.

In the directory, *CodeSniffer* → *Standards*, all the coding standards are defined separately. However, in the directory, *CodeSniffer* → *Standards* → *PEAR* → *Sniffs*, all the sniffs are grouped by the sniff type.

To create your own sniff, follow the same directory structure and create the following file:

```
Standards/PROJECT_A/Sniffs/LocalSettings$ vim
DisallowLocalSettingsSniff.php
```

While writing your own sniff, remember the following points:

- Implement the interface *PHP_CodeSniffer_Sniff*
- Use the register method to pick the token you are interested in sniffing
- Use the process method to check and raise the error

```
<?php
Class ProjectA_Sniffs_LocalSettings_DisallowLocalSettingsSniff
implements PHP_CodeSniffer_Sniff
{
    public function register()
    {
        return array(T_COMMENT);
    }

    public function process(PHP_CodeSniffer_File $phpcsFile,
$stackPtr)
    {
        $tokens = $phpcsFile->getTokens();
        //check for the content
        if (trim($tokens[$stackPtr]['content']) === '//test_
code') {
            $error = 'Test code exist. .Please Remove it. found
%';

            //Take the next line of the comment
            $data = array(trim($tokens[$stackPtr + 1]
['content']));
            $phpcsFile->addError($error, $stackPtr + 1, 'Found',
$data);
        }
    }
}
```